AN ENGINEERING PROJECT REPORT

On

ADAPTIVE RENDERING ENGINE

Submitted By

Bijay Bk – 220305

Devendra Pandey – 220306

Manish Joshi – 220312

Pramod Panta – 220317

Submitted to

The Department of ICT and Computer Engineering

In Partial fulfillment of the requirements for the degree of

Bachelor of Engineering in Computer

Cosmos College of Management & Technology

(Affiliated to Pokhara University)

Sitapaila, Kathmandu, Nepal

Date of Submission: Bhadra 2083 B.S. (August 2026 A.D.)

AN ENGINEERING PROJECT REPORT

On

ADAPTIVE RENDERING ENGINE

Submitted By

Bijay Bk – 220305

Devendra Pandey – 220306

Manish Joshi – 220312

Pramod Panta – 220317

Under the Supervision of

Er. Robinhood Khadka

Submitted to

The Department of ICT and Computer Engineering

In Partial fulfillment of the requirements for the degree of

Bachelor of Engineering in Computer

Cosmos College of Management & Technology

(Affiliated to Pokhara University)

Sitapaila, Kathmandu, Nepal

Date of Submission: Bhadra 2083 B.S. (August 2026 A.D.)

COPYRIGHT

CERTIFICATE

The undersigned certify that they have read and recommended to the Department of ICT and Computer Engineering, a final year project work entitled "Adaptive Rendering Engine" submitted by Bijay Bk (220305), Devendra Pandey (220306), Manish Joshi (220312) and Pramod Panta (220317) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer.

_____

Er. Robinhood Khadka
(Project Supervisor)
Department of ICT and Computer Engineering
Cosmos College of Management & Technology

_____

(External Examiner)

_____

Head of the Department
Department of ICT and Computer Engineering
Cosmos College of Management & Technology

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project supervisor, Er. Robinhood Khadka, for his continuous guidance, encouragement and valuable feedback throughout the design, implementation and evaluation of the Adaptive Rendering Engine. His insistence that every claim be demonstrable shaped the evidence-first character of this work.

We are equally thankful to the Department of ICT and Computer Engineering, Cosmos College of Management & Technology, for providing the academic environment and the resources required to carry out this project, and to our teachers and friends for their reviews and criticism at every stage.

We also acknowledge the open-source communities behind Node.js, React, Docker, nginx and the wider web-performance ecosystem, whose freely available tools made it possible to build and rigorously evaluate this project entirely at zero cost. Finally, we thank our families for their constant support.

Bijay Bk (220305)
Devendra Pandey (220306)
Manish Joshi (220312)
Pramod Panta (220317)

ABSTRACT

Modern web applications are built on a family of rendering strategies — Static Site Generation (SSG), Server-Side Rendering (SSR), Client-Side Rendering (CSR), Incremental Static Regeneration (ISR), Streaming SSR and edge rendering — each of which trades latency, freshness, server cost and client work against the others. In every mainstream framework, including Next.js and Remix, the choice among them is made by a developer at build time and then applied uniformly to every request. A route marked SSR is rendered identically for a low-end handset on a congested 2G link and for a desktop workstation on fibre, even though the optimal answer for those two visitors is not the same.

This project designs, implements and evaluates an Adaptive Rendering Engine (ARE): a runtime that removes the rendering strategy from build-time configuration and turns it into a per-request decision. For every incoming request the engine observes five contextual signals — network speed, device class, cache state, server load and data volatility, together with a payload-weight flag and an edge flag — and evaluates a pure, ordered, first-match-wins rule table to select one of six pluggable strategy modules. The selected strategy and the exact rule that produced it are returned on every response as the X-Rendering-Strategy and X-Decision-Reason headers and written to the server log, so the behaviour of the engine is externally observable rather than merely asserted. The system is built on Node.js 20, TypeScript and React 18 over the native HTTP module, and is deployed on a zero-cost, Docker-based private server that models one origin, two latency-injecting edge nodes, an nginx reverse proxy and a shared Redis cache.

The engine was evaluated on that private server. All 17 documented context-to-strategy triggers reproduce exactly; 28 automated tests pass; and an exhaustive enumeration of the 648-point context space shows that the rule table is total, deterministic and free of unreachable rules, while a decision costs 26.53 ns (37.7 million decisions per second), roughly one ten-thousandth of the measured request cost. Against a fixed SSR policy under 800 requests at concurrency 50, adaptive selection raised throughput from 1,458.7 to 3,172.2 requests per second (+117.5 %), cut mean latency by 54.0 % and 99th-percentile latency by 86.6 %, and reduced the transferred document from 7,905 to 853 bytes on a fast link; on a 100 ms link, where the network rather than the origin is the bottleneck, throughput was unchanged (-0.7 %) but origin CPU fell by 34.0 % and tail latency by 8.1 %. The engine therefore demonstrates, with reproducible evidence, that rendering-strategy selection is a legitimate runtime optimisation variable, and it provides an extensible platform on which learned and predictive selection policies can later be built.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS / ABBREVIATIONS

ARE	Adaptive Rendering Engine
SSG	Static Site Generation
SSR	Server-Side Rendering
CSR	Client-Side Rendering
ISR	Incremental Static Regeneration
EDGE_ISR	Edge Incremental Static Regeneration
SWR	Stale-While-Revalidate
TTFB	Time To First Byte
FCP	First Contentful Paint
LCP	Largest Contentful Paint
TTL	Time To Live
HTTP	HyperText Transfer Protocol
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
NDJSON	Newline-Delimited JSON
CDN	Content Delivery Network
API	Application Programming Interface
SEO	Search Engine Optimization
UA	User-Agent
TS	TypeScript
ab	ApacheBench, the Apache HTTP server benchmarking tool

1. Introduction

1.1 Introduction to the Project

The Adaptive Rendering Engine (ARE) is a web runtime that decides, separately for every single HTTP request it serves, how the requested page should be produced. It is deliberately built as technology — an engine — and not as an end-user application. It has no business domain, no user accounts and no database; what it has is a decision pipeline, six interchangeable rendering back-ends and an evidence trail.

Concretely, the engine receives a request, observes the conditions under which that request arrived, selects one of six rendering strategies (SSG, SSR, Streaming SSR, ISR, CSR or a simulated Edge-ISR), renders the page with the selected strategy, and reports both the choice and the reason for the choice back to the caller in the response headers. The same URL, requested twice under different conditions, is legitimately answered in two different ways. That single sentence is the entire contribution of the project, and everything in this report exists either to build it or to prove it.

The whole system runs on free and open-source software on a laptop. There is no cloud account, no paid hosting, no domain name and no public IP address anywhere in the project. The execution environment is a Docker Compose stack that models a small content delivery topology — an origin server, two edge nodes with different injected latencies, a reverse proxy and a shared cache — entirely on the local machine.

1.2 Origin of the Idea and the Vision

The idea did not begin with a literature search. It began with an observation made while building an ordinary Next.js site: the framework asks the developer to annotate each route with how it should be rendered, and it asks this question at the one moment when the answer cannot possibly be known — before any user has arrived. A route annotated for server-side rendering is server-rendered for the visitor on a fibre-connected desktop who would have been better served a light interactive shell, and equally for the visitor on a throttled mobile link who genuinely needs finished HTML. The annotation is a single bet placed on an average user who does not exist.

Reframing that observation produced the project. If the rendering strategy is a variable rather than a constant, then choosing it is an optimisation problem; and if it is an optimisation problem, it belongs at runtime, where the inputs are actually available. The vision that follows is stated once here and is the standard against which the rest of this report should be read:

> *Rendering strategy should be a runtime decision computed from the conditions of the request, not a build-time constant chosen by a developer; and every such*

> *decision should be observable, explainable and reproducible from outside the system.*

Three commitments follow from that vision and shaped every design choice made in the project. First, the decision must be honest: the engine must publish what it decided and why, on every response, so that a reviewer can verify the behaviour without reading the source code. Second, the decision must be cheap: an adaptive engine that spends more time deciding than rendering has defeated itself, so the selection logic is a pure function over primitive values with no input/output of any kind. Third, the decision must be extensible: adding a seventh strategy, or replacing the rule table with a learned model, must not require editing any existing strategy.

A fourth, quieter commitment emerged during implementation and deserves to be recorded as part of the vision rather than as an afterthought: the demonstration pages themselves should explain the engine. Rather than shipping a generic sample page, each demonstration page renders the observed context, the rule that fired and the resulting strategy into its own markup, so the system is self-documenting when opened in a browser with JavaScript disabled.

1.3 Background

Server-side rendering was the original model of the web: the server produced complete HTML and the browser displayed it. The rise of rich client applications inverted that model into client-side rendering, in which the server ships an empty shell and a JavaScript bundle constructs the interface in the browser. Each inversion solved the previous model's problem and created a new one — CSR restored interactivity at the cost of a slow first paint on weak devices and links, and SSR restored the fast first paint at the cost of server CPU under load.

Subsequent strategies are attempts to occupy the space between those poles. Static Site Generation renders once at build time and serves a file, which is unbeatably cheap but cannot express fresh data. Incremental Static Regeneration keeps the cheap cached response but bounds its staleness with a time-to-live and a background refresh. Streaming SSR sends the page shell as soon as it exists and streams the remainder as it resolves, decoupling time-to-first-byte from total render time. Edge rendering moves any of the above physically closer to the user.

The important structural fact is that these six strategies are not competitors to be ranked; they are points on a trade-off surface whose optimum moves with the request. Each is optimal somewhere. The literature reviewed in Chapter 2 measures them thoroughly but almost always compares them in isolation, as fixed architectural choices, rather than treating the choice itself as a controllable runtime variable — which is precisely the gap this project occupies.

1.4 Problem Statement

Rendering strategy selection in modern web applications is static and developer-defined. It is fixed at development time, applied uniformly to every request thereafter, and cannot react to the runtime conditions that determine whether the choice was correct — network speed, device capability, cache state, server load or data volatility. This produces three concrete deficiencies: sub-optimal response characteristics whenever conditions differ from the developer's assumption, inefficient use of origin resources when a cheaper strategy would have sufficed, and an inconsistent experience across a heterogeneous population of devices and networks.

No unified runtime mechanism exists at the application layer that combines contextual analysis with rendering orchestration, and consequently there is no experimental platform on which the value of such adaptation can be measured. The problem addressed by this project is therefore stated as:

> *Rendering strategy selection in modern web applications is static and does not adapt to runtime contextual conditions, resulting in reduced performance efficiency, avoidable origin cost and scalability limitations.*

1.5 Objectives

1.5.1 General Objective

To design, implement and evaluate an Adaptive Rendering Engine that selects a web rendering strategy per request at runtime from observed contextual variables, and to demonstrate experimentally that doing so improves performance and resource efficiency relative to a fixed rendering policy.

1.5.2 Specific Objectives

1. To identify and formally define the contextual variables that influence rendering performance, and to build an analyzer that observes them from a live request.
2. To design a pure, deterministic, rule-based decision engine that maps any context to exactly one rendering strategy, and to prove that mapping total and unambiguous.
3. To implement six rendering strategies — SSG, SSR, Streaming SSR, ISR, CSR and a simulated Edge-ISR — behind a single uniform interface so that they are interchangeable at runtime.
4. To make every decision externally observable through response headers, structured server logs and per-request metrics.
5. To construct a zero-cost, reproducible, Docker-based private server that models an origin, multiple edge nodes and a shared cache.

6. To measure per-request cost, payload size, cache efficiency and resource utilisation, and to compare adaptive selection against fixed single-strategy policies under controlled load.

1.6 Scope and Limitations of the Project

The project covers the design and implementation of a standalone adaptive rendering runtime; the simulation of network classes, device profiles, cache states and server load within a local containerised environment; the instrumentation required to measure the engine; and a controlled comparative evaluation of adaptive selection against fixed policies.

The following are explicitly outside scope. The engine is not deployed to commercial cloud infrastructure; the edge topology is modelled locally with injected latency rather than geographically distributed. It is not a production-scale web application and carries no business domain. Client-side paint metrics such as First Contentful Paint and Largest Contentful Paint are not instrumented; the measurements reported here are server-side timings, transferred bytes and process resource counters, which are the quantities the engine itself can influence and observe. Finally, the decision policy is rule-based by design; machine-learned policies are identified as future work in Chapter 7 and the architecture is deliberately shaped to accommodate them.

1.7 Team Members and Plan Followed

Table 1.1: Project Team Members and Primary Responsibilities

Name	Roll No.	Primary Responsibility
Bijay Bk	220305	Core engine, context analyzer, decision engine, HTTP server
Devendra Pandey	220306	Rendering strategy modules and React page components
Manish Joshi	220312	Cache subsystem, metrics pipeline, Docker private server
Pramod Panta	220317	Testing, experimental validation, documentation

The team followed an incremental, milestone-driven plan with weekly checkpoints, Git version control and periodic supervisor reviews. The plan was organised in five phases; Table 1.2 records the phases and their final status.

Table 1.2: Project Phases and Final Status

Phase	Activity	Status
1	Research consolidation, architecture and decision-rule design	Completed
2	Core engine, six strategy modules, cache and metrics subsystems	Completed
3	Containerised private server: origin, two edges, proxy, Redis	Completed
4	Experimental testing, controlled measurement and data collection	Completed
5	Analysis, comparative evaluation and final documentation	Completed

1.8 Application of the Project

The engine is a runtime component, so its applications are architectural rather than end-user facing:

- Framework-level optimisation. The analyze-decide-render pipeline can be embedded as a middleware layer in an existing Node.js application to make rendering adaptive without rewriting the application's pages.
- Low-bandwidth and low-end-device populations. Educational, governmental and public-service sites serving a wide device mix can automatically fall back to finished HTML with minimal JavaScript for weak clients while giving capable clients a fully interactive shell.
- Traffic-spike resilience. When the load classifier observes elevated concurrency, the engine shifts to cache-backed strategies and sheds origin work, which the evaluation in Section 5.8 measures as a 34.0 % reduction in origin CPU.
- Content delivery research. Because every decision is logged with its full context and timings, the system doubles as an instrument for studying rendering trade-offs empirically.
- Teaching. The self-explaining demonstration pages make the six strategies, and the difference between them, directly visible in a browser.

1.9 Organisation of the Report

Chapter 2 reviews the literature on rendering strategies, performance measurement and context-aware adaptation, and states the research gap. Chapter 3 presents the system analysis, the design philosophy and the architecture. Chapter 4 documents the methodology and the implementation, including a walkthrough of the request lifecycle in code and the decision algorithm in detail. Chapter 5 reports the experimental results, the performance analysis and the comparative evaluation. Chapter 6 discusses what the results mean, the problems encountered during development and the limits of validity. Chapter 7 concludes and sets out future enhancements.

2. Literature Review

This chapter surveys what is already known about web rendering strategies, how their performance is measured, and where context-aware adaptation has been applied successfully. The purpose is to establish precisely which part of the problem is already solved and which part is not.

2.1 Web Rendering Strategies and Their Trade-offs

Comparative studies of rendering methods establish the basic trade-off surface. Server-side rendering improves initial page-load performance and search-engine visibility because content is fully materialised before it reaches the browser, but it raises server workload, particularly under concurrent traffic [2], [4], [5]. Client-side rendering inverts this: it reduces server overhead and yields excellent interactivity, but delays first paint on weak networks and constrained devices [3], [9]. Hybrid strategies attempt to combine the advantages: static generation and incremental static regeneration pre-produce content to cut server strain and improve cache efficiency, accepting bounded staleness in return [5], [6].

Two observations from this body of work are load-bearing for the present project. The first is that no strategy dominates; each is optimal in some region of the condition space. The second is methodological: these studies almost uniformly evaluate strategies in isolation, as fixed architectural properties of a site, rather than as alternatives that could be selected between dynamically.

2.2 Performance Metrics in Web Applications

Web-performance research converges on a small set of measurable indicators: Time to First Byte, First Contentful Paint, Largest Contentful Paint, CPU utilisation, memory consumption and cache-hit ratio [7], [8]. Studies of application performance identify front-end complexity, inefficient asset loading and JavaScript execution overhead as the dominant bottlenecks [7], [8], and mobile-focused work stresses perceived performance under constrained connectivity [9], [10]. These works supply a robust measurement vocabulary, which this project adopts; what they do not do is treat the rendering strategy itself as an independent variable that a system could manipulate to move those metrics.

2.3 Server-Side Rendering and Search-Engine Visibility

Work evaluating SSR specifically confirms its positive effect on interface responsiveness and crawlability, since search-engine crawlers receive materialised markup rather than an empty shell [4]. The same experimental analyses, however, show elevated CPU and memory consumption under high concurrency [6]. The conclusion drawn in the literature — that SSR is context-dependent rather than universally optimal — is exactly the premise on which an

adaptive selector is justified.

2.4 Caching, Scalability and Edge Placement

Caching is a well-established scalability mechanism; controlled studies of server-side caching report substantial latency reductions and throughput gains [6]. Mobility-aware edge caching research goes further and demonstrates that context-aware placement of content reduces both response time and network congestion [11]. This is significant because it establishes the principle that contextual adaptation improves performance. The limitation is one of layer: the adaptation occurs at the network or infrastructure tier, deciding where content is stored, and never at the application tier, deciding how content is produced.

2.5 Device and Browser-Level Considerations

Mobile web-performance research highlights how sensitive rendering outcomes are to bandwidth variability and device capability [9], [10], and analyses of in-browser computation document the growing execution cost borne by the client [1]. Together these motivate treating device class as a first-class decision input — a rendering decision that ignores whether the client can afford to hydrate a large payload is incomplete. Current frameworks, however, provide no mechanism for adjusting the strategy on the basis of such signals at request time.

2.6 Limitations of Existing Research and the Identified Gap

Summarising, the reviewed literature supplies comparative evaluations of rendering strategies [2], [5], empirical analysis of SSR performance and SEO impact [4], caching and edge-placement optimisation [6], [11], performance benchmarking frameworks [7], [8], and mobile performance studies [9], [10]. Four limitations persist across it, and Table 2.1 maps each to the response made by this project.

Table 2.1: Limitations in the Reviewed Literature and the Response of This Project

Limitation identified in the literature	Response implemented in this project
Rendering strategies are treated as static architectural decisions	Strategy is a runtime variable recomputed for every request
Contextual variables are rarely integrated into rendering selection	Seven contextual signals are observed and drive an explicit rule table
No unified runtime engine orchestrates rendering strategies	A single engine registers six strategies behind one interface and selects among them
Experimental validation of adaptive rendering at the application layer is limited	A reproducible private server, 28 automated tests and a controlled comparative evaluation against fixed policies

The gap is therefore precise. Prior work establishes that rendering strategies differ measurably, that contextual adaptation is valuable, and how performance should be measured;

it does not propose, implement or experimentally validate a runtime engine that unifies contextual analysis with rendering orchestration at the application layer. This project addresses that gap directly, and Chapter 5 supplies the experimental validation the literature lacks.

3. System Analysis and Design

This chapter states what the system had to do, the design principles adopted to do it, and the architecture that resulted. Chapter 4 then describes how that architecture was built.

3.1 Requirement Analysis

3.1.1 Functional Requirements

1. The system shall observe, for every request, the network class, device class, cache state, server load, data volatility, payload weight and edge status applicable to that request.
2. The system shall select exactly one rendering strategy per request from these observations.
3. The system shall support six strategies — SSG, SSR, Streaming SSR, ISR, CSR and Edge-ISR — and shall render the response using the selected one.
4. The system shall report the selected strategy and the reason for the selection on every response and in the server log.
5. The system shall allow every contextual signal to be overridden externally, so that any context can be reproduced on demand for testing and demonstration.
6. The system shall record per-request timing, payload and resource metrics to a durable log and aggregate them into per-strategy summaries.

3.1.2 Non-Functional Requirements

1. Determinism. Identical contexts shall always produce identical strategy selections.
2. Totality. Every reachable context shall map to a strategy; selection shall never fail.
3. Negligible overhead. The cost of deciding shall be insignificant beside the cost of rendering.
4. Fault tolerance. A failure inside any strategy shall degrade to a correct response rather than an error.
5. Extensibility. Adding a strategy or changing the policy shall not require modifying existing strategies.
6. Zero cost and reproducibility. The entire system shall run offline on commodity hardware using only open-source components.

3.2 Design Philosophy

Five invariants were fixed before implementation began. They are stated here because every later design decision is an application of one of them, and because Chapter 5 measures

whether they were achieved.

Separation of concerns. The pipeline is split into stages that are forbidden to do each other's work: analysis observes but never decides; decision selects but never renders; rendering produces output but never re-derives the decision. This is why the decision can be tested without a server and why a strategy cannot silently disagree with the engine about why it was invoked.

Purity of the decision. Strategy selection is implemented as a pure function from a plain context object to a decision record, performing no input/output, consulting no clock and holding no state. Purity is not stylistic here: it is what makes the selection provably deterministic, exhaustively enumerable (Section 5.3) and testable at the granularity of a single rule.

Pluggability. All six strategies implement one interface and are registered into a registry at start-up. The engine holds a reference to the registry, not to any strategy. Adding a seventh strategy is an insertion into the registry and a rule in the table; no existing strategy module is touched.

Transparency. A decision that cannot be observed cannot be trusted. Every response carries the strategy and the human-readable rule that produced it; every request emits three log lines recording the URL, the observed context and the selection; every request appends a metrics record. The engine is therefore auditable from outside.

Graceful degradation. The rule table terminates in an unconditional fallback so selection cannot fail, and the render call is wrapped so that a strategy that throws is replaced by SSR, with the substitution stated in the reason string rather than hidden.

3.3 System Architecture

Figure 3.1 shows the deployed architecture. A single nginx reverse proxy is the only entry point on port 8080. It routes the default path to the origin node and the /edge1 and /edge2 prefixes to two edge nodes, which are also published directly on ports 8081 and 8082. Every node — origin and both edges — runs the identical engine image and differs only by environment variables: an identity (SERVED_BY), an injected latency (EDGE_LATENCY_MS) and a cache time-to-live. A Redis container provides an optional cache shared between the edges. All five containers share one private Docker network and address each other by service name, so the topology requires no public address of any kind.

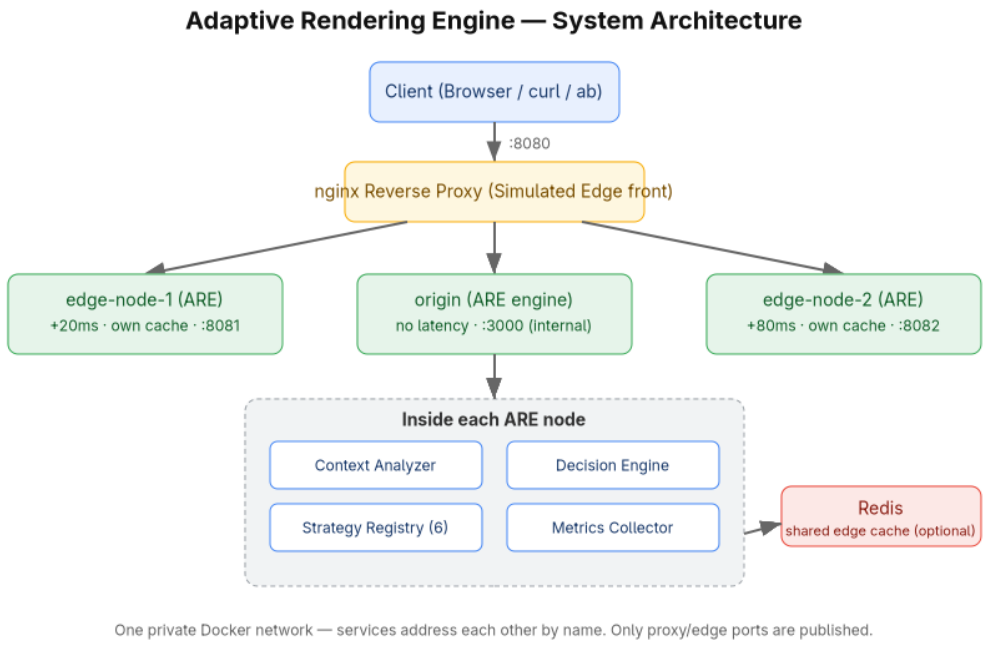

Figure 3.1: High-Level System Architecture of the ARE Private Server

Modelling the edges as full engine instances rather than as caching proxies was a deliberate decision. An edge that is only a proxy can demonstrate distance but not behaviour; an edge that runs the engine has its own context (it self-identifies as an edge), its own cache namespace and its own decisions, which is what makes the Edge-ISR strategy meaningful rather than decorative.

3.4 The Rendering Pipeline

Internally, every request traverses the same five-stage pipeline, shown in Figure 3.2. The stages are analyze, decide, render, respond and measure. Stage boundaries are strict: the output of each stage is a value, and the next stage consumes only that value.

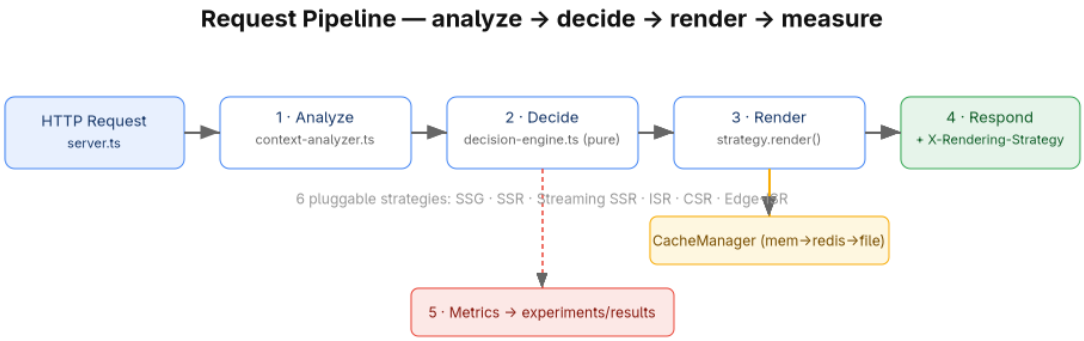

Figure 3.2: Per-Request Rendering Pipeline

- ANALYZE (src/core/context-analyzer.ts) turns the raw request into a RequestContext. It resolves each signal by precedence and validates it, but it makes no decisions.

- DECIDE (src/core/decision-engine.ts) applies the rule table to the context and returns a DecisionTrace holding the selected strategy, the reason and the context it judged.
- RENDER (src/strategies/...) looks the selected strategy up in the registry and invokes it. The reason travels with the call so the page can state why it exists in that form.
- RESPOND (src/core/engine.ts) writes the status and headers, always adding X-Rendering-Strategy and X-Decision-Reason, then sends the body — a string for buffered strategies or a piped stream for Streaming SSR.
- MEASURE (src/metrics/metrics-collector.ts) appends a metrics record. The call is not awaited, so instrumentation can never delay a response.

3.5 Component Design

This section describes each component of the engine and the role it plays. Implementation detail is deferred to Chapter 4.

3.5.1 Context Analyzer

The analyzer is the system's sensory layer. It produces a RequestContext containing seven decision-relevant fields plus the URL and raw headers. For each field it applies a fixed precedence — an explicit control header, else a query-string alias of that header, else inference from the request or from live server signals — and validates the result against the permitted value set, so an unrecognised value falls back to inference rather than corrupting the context. Load is not supplied by the request at all: it is classified from a live counter of in-flight requests maintained by the server. Cache state is probed from the cache layer before analysis begins.

3.5.2 Decision Engine

The decision engine holds the policy. It evaluates an ordered table of rules against the context, top to bottom, and returns the first rule that matches together with its human-readable reason. Because the final rule is unconditional, the function is total. Because it touches nothing outside its argument, it is pure. The policy itself lives in a separate configuration module, so the policy can be edited without touching the evaluator.

3.5.3 Strategy Registry and Strategy Modules

The registry is a name-to-implementation map populated at start-up. Each of the six strategies implements a single render method with an identical signature, receiving the context, the page module, the cache manager and the decision reason, and returning a status, headers, a body and a flag stating whether the response came from cache. Uniformity of that signature is what makes the strategies genuinely interchangeable.

3.5.4 Cache Manager

The cache manager presents one interface over three backends arranged in a hierarchy: an in-process memory cache, an optional shared Redis cache and a persistent filesystem cache. Reads descend memory, then Redis, then file, and promote any hit upward so subsequent reads are faster; writes fan out to all available backends. Redis is optional by construction — if it is absent the manager logs a warning and continues on memory and file alone. The manager also exposes a non-mutating freshness probe, which is what allows the analyzer to observe cache state before a decision is made.

3.5.5 Metrics Collector and Report Generator

The collector appends one newline-delimited JSON record per request, capturing timings, payload size, cache outcome, the observed context and a process resource sample. The report generator reads that log and aggregates it per strategy into JSON and CSV summaries. All quantitative results in Chapter 5 derive from this pipeline or from external measurement of the same requests.

3.5.6 Simulation Subsystem

Because the evaluation runs on one machine, conditions must be created rather than awaited. The network throttler applies a server-side delay of 400 ms, 100 ms or 0 ms for the slow, medium and fast classes. Edge latency is injected per container from an environment variable. Load is not simulated but genuinely measured from concurrency. Device class, cache state and volatility are supplied through the control surfaces.

3.5.7 Demonstration Pages

Three React pages exercise the engine: a static page, a realtime page and a heavy page. Each declares its own volatility and payload weight, which become inputs to the decision, and each renders the engine's observed context and selected rule into its own markup so the decision is visible in the page source.

3.6 Data Design

Five data structures carry all state between stages. Their fields are the vocabulary in which the entire system is written.

Table 3.1: Core Data Structures of the Engine

Structure	Purpose and principal fields
RequestContext	Output of ANALYZE. url, networkSpeed (slow|medium|fast), device (mobile|desktop), cacheState (fresh|stale|cold), load (low|medium|high), volatility (static|periodic|realtime), heavyPayload, isEdge, rawHeaders
DecisionTrace	Output of DECIDE. selected (strategy name), reason (rule text), context
RenderStrategy	The uniform strategy interface. name, and render(ctx, page, cache, meta)
RenderResult	Output of RENDER. status, headers, body (string or stream), fromCache
MetricRecord	Output of MEASURE. ts, url, strategy, reason, ttfbMs, renderMs, totalMs, fromCache, bytes, network, device, load, isEdge, resources

3.7 Control Surfaces

Every contextual signal can be set explicitly. This is what converts an adaptive system, which would otherwise be difficult to test because its behaviour depends on conditions, into an experimentally controllable one. Two equivalent layers exist, with headers taking precedence over query parameters and both taking precedence over inference.

Table 3.2: Control Surfaces for the Contextual Signals

Signal	Control header	Query alias	Permitted values
Network speed	X-Network-Speed	?net=	slow | medium | fast
Device class	X-Device-Type	?device=	mobile | desktop
Cache state	X-Cache-State	?cache=	fresh | stale | cold
Server load	X-Load-Level	?load=	low | medium | high
Data volatility	X-Data-Volatility	?volatility=	static | periodic | realtime
Payload weight	X-Data-Size	?size=	heavy | light
Edge identity	X-Served-By	?served=	any value other than 'origin' means edge

The query-parameter layer exists for a specific reason. A browser cannot attach custom headers to an ordinary navigation, so without query aliases the in-page controls could only predict which strategy would be chosen. With them, following a link genuinely re-renders the page under a different strategy, which is what makes the system demonstrable in a browser rather than only through a command-line client.

3.8 Deployment Design

Table 3.3: Docker Service Topology of the Private Server

Service	Role	Published port	Key environment
proxy	nginx reverse proxy, single entry point	8080 -> 80	routes /, /edge1/, /edge2/
origin	Engine, authoritative node	internal only	SERVED_BY=origin, EDGE_LATENCY_MS=0, TTL=30 s
edge-node-1	Engine as a near edge	8081 -> 3000	SERVED_BY=edge-node-1, +20 ms, TTL=15 s
edge-node-2	Engine as a far edge	8082 -> 3000	SERVED_BY=edge-node-2, +80 ms, TTL=15 s
redis	Shared cache across nodes	internal only	in-memory only, no persistence

Two consequences of this design are worth recording because they shape how the system is operated. The proxy sets X-Served-By to 'origin' on the default route, which correctly prevents a client from claiming to be at an edge when it is not, but which also means the Edge-ISR strategy must be exercised through the /edge1 and /edge2 routes or against the edge ports directly. And because the stack mounts no volumes, the container filesystem is the only store for generated artefacts, caches and metrics; a rebuild is therefore required to deploy changed code, and a teardown discards accumulated state, which is precisely what makes each experimental run start from a known cold state.

4. Methodology and Implementation

4.1 Development Methodology

The project follows a design-and-build methodology combined with a quantitative experimental evaluation. The build phase produced an artefact; the evaluation phase treated that artefact as an experimental apparatus. In the experimental model the independent variables are the contextual signals (network speed, device class, cache state, load and data volatility), the moderating variable is the rendering strategy that the engine selects, and the dependent variables are server-side time to first byte, render time, response size, cache-hit rate, throughput and process resource consumption. Controlled simulation is used to hold all but one variable constant, which is only possible because every signal has an explicit control surface (Section 3.7).

4.2 Technology Stack and Justification

Table 4.1: Technology Stack and Justification

Concern	Technology	Justification
Language	Node.js 20+ with TypeScript	One language across engine and view; static types make the context and rule structures verifiable at compile time
View layer	React 18	The only mainstream library exposing all the primitives the six strategies need: renderToString, renderToPipeableStream with Suspense, and hydrateRoot
HTTP server	Native Node http module	The deliverable is itself a runtime; a framework would hide the very request handling being studied, and native streaming is required by Streaming SSR
Client bundler	esbuild	Produces the hydration and CSR bundle with no configuration and negligible build time
Cache	Memory + filesystem + optional Redis	Zero-cost persistence, plus a genuinely shared cache to make edge behaviour real
Edge and proxy	nginx in containers	Provides a single entry point and models edge routing
Orchestration	Docker and Docker Compose	Reproducible multi-node topology on one machine at no cost
Testing	Vitest	TypeScript-native and fast enough to run on every change
Load testing	Apache Bench	Standard concurrent-request generator with percentile reporting

React is used strictly as a rendering primitive. The contribution of this project is the selector built around it, not a new view library, and no part of React was modified.

4.3 The Request Lifecycle in Code

This section traces one request through the source, because the flow of control is the clearest statement of how the system actually works.

4.3.1 Entry and pre-processing

The server (src/server/server.ts) creates a native HTTP server. On entry it increments an in-flight counter — the sole source of the load signal — inside a try/finally so the counter is decremented on every exit path, including errors. It then serves static assets, the client bundle, the data endpoint and the health endpoint directly; only requests that resolve to a page module continue into the engine.

Two pre-processing steps follow. First, if the node is running as an edge and the client did not supply an identity, the container stamps its own SERVED_BY value onto the headers, so the analyzer will observe isEdge. Second, the simulated conditions are applied: the server sleeps for the delay corresponding to the requested network class and then for the container's edge latency. Both sleeps occur before the engine is invoked, which is the reason server-side timings in Chapter 5 measure engine cost alone and are not contaminated by the simulated link.

```
  const server = http.createServer(async (req, res) => {
    inFlight++;
    try {
      ...
      if (!headers['x-served-by'] && config.servedBy)
        headers['x-served-by'] = config.servedBy;

      const speed = headers['x-network-speed'] ?? queryOf(req).get('net') ?? 'medium';
      await sleep(delayForNetwork(speed));
      if (config.edgeLatencyMs > 0) await sleep(config.edgeLatencyMs);

      const cacheState = await peekCacheState(page);
      await engine.handle({ url, headers, page,
                            signals: { concurrency: inFlight, cacheState }, res });
    } finally { inFlight--; }
  });
```

4.3.2 Stage 1 — analyze

The analyzer resolves each signal through a single helper that implements the precedence rule, and validates the result against the permitted set. An unrecognised value is discarded rather than propagated, so a malformed header degrades to inference instead of producing an invalid context. Load is classified from the concurrency counter at thresholds of 25 for high and 8 for medium; device is inferred from the User-Agent when not stated; volatility defaults to the value the page declares about itself.

```
  function override(headers, query, headerName) {
    return header(headers, headerName) ?? query.get(QUERY_ALIASES[headerName]) ?? undefined;
  }
  function oneOf(value, allowed) {
    return allowed.includes(value) ? value : undefined;   // invalid -> fall back to inference
  }

  return {
    url,
```

```
  networkSpeed: oneOf(pick('x-network-speed'), ['slow','medium','fast']) ?? 'medium',
  device:       oneOf(pick('x-device-type'), ['mobile','desktop']) ?? inferDevice(headers),
  cacheState:   oneOf(pick('x-cache-state'), ['fresh','stale','cold']) ?? signals.cacheState,
  load:         oneOf(pick('x-load-level'), ['low','medium','high'])
                  ?? classifyLoad(signals.concurrency),
  volatility:   oneOf(pick('x-data-volatility'), ['static','periodic','realtime'])
                  ?? page.volatility,
  heavyPayload: size ? size.toLowerCase() === 'heavy' : page.heavy === true,
  isEdge:       Boolean(servedBy && servedBy !== 'origin'),
  rawHeaders:   headers,
};
```

4.3.3 Stage 2 — decide

The evaluator is nine lines long, and its brevity is the point: all policy lives in data, so the code that applies the policy has nothing to get wrong.

```
export function decide(ctx: RequestContext): DecisionTrace {
  for (const rule of STRATEGY_RULES) {
    if (rule.test(ctx)) {
      return { selected: rule.strategy, reason: rule.reason, context: ctx };
    }
  }
  return { selected: 'SSR', reason: 'Fallback (no rule matched)', context: ctx };
}
```

4.3.4 Stages 3 to 5 — render, respond, measure

The engine looks the selected strategy up in the registry and invokes it inside a try/catch. If the strategy throws, the engine renders with SSR instead and rewrites the reason to state that a substitution occurred, so a degraded response is never silently indistinguishable from a healthy one. It then merges the strategy's headers with the two proof headers. Because HTTP header values must be Latin-1, the reason string is sanitised of non-ASCII characters on the way out while the log retains the original text.

Timing is taken at two points: time to first byte is captured immediately after the headers are written, and total time after the body has been flushed, with render time derived as the difference. For streaming responses the engine counts bytes as chunks pass and resolves when the stream ends, so a streamed response is measured on the same basis as a buffered one. The metrics write is issued without awaiting it.

```
let result: RenderResult;
try {
  result = await this.deps.registry.get(trace.selected)
                    .render(ctx, page, this.deps.cache, { reason: trace.reason });
} catch (err) {
  result = await this.deps.registry.get('SSR')
                    .render(ctx, page, this.deps.cache,
                            { reason: `${trace.selected} render failed - fell back to SSR` });
}

const headersOut = { ...result.headers,
  'x-rendering-strategy': trace.selected,
  'x-decision-reason'  : trace.reason.replace(/[^\x20-\x7E]/g, '-') };
```

```
  res.writeHead(result.status, headersOut);
  const ttfbMs = timer.elapsedMs();
  ...
  void this.deps.metrics.record({ ... });     // never awaited: metrics cannot delay a response
```

4.4 The Decision Algorithm

The decision algorithm is the intellectual centre of the project and is specified here in full. It is an ordered, first-match-wins rule table over the context. Table 4.2 is the authoritative policy as implemented, and Figure 4.1 shows the same policy as a flow.

Table 4.2: Decision Rule Table (Context to Strategy), evaluated top to bottom

#	Condition	Strategy	Rationale
1	volatility = static AND cache != cold	SSG	Static content with a usable cache: serve the pre-built artefact
2	volatility = static AND isEdge	EDGE_ISR	Static content at an edge: revalidate close to the user
3	load = high	ISR	Shed origin work: serve cached output and revalidate in the background
4	volatility = realtime AND net = fast AND device = desktop	CSR	Capable client on a fast link: ship a shell and let it be fully interactive
5	volatility = realtime AND device = mobile	SSR	Weak device: send finished HTML and minimal JavaScript
6	volatility = periodic	ISR	Periodically changing data: cache and revalidate on a TTL
7	heavyPayload AND net != slow	STREAMING_ SSR	Large payload on a decent link: stream the shell first, then the rest
8	net = slow	SSR	Slow link: avoid the cost of hydrating a large bundle
9	unconditional fallback	SSR	Safe, correct default that makes the function total

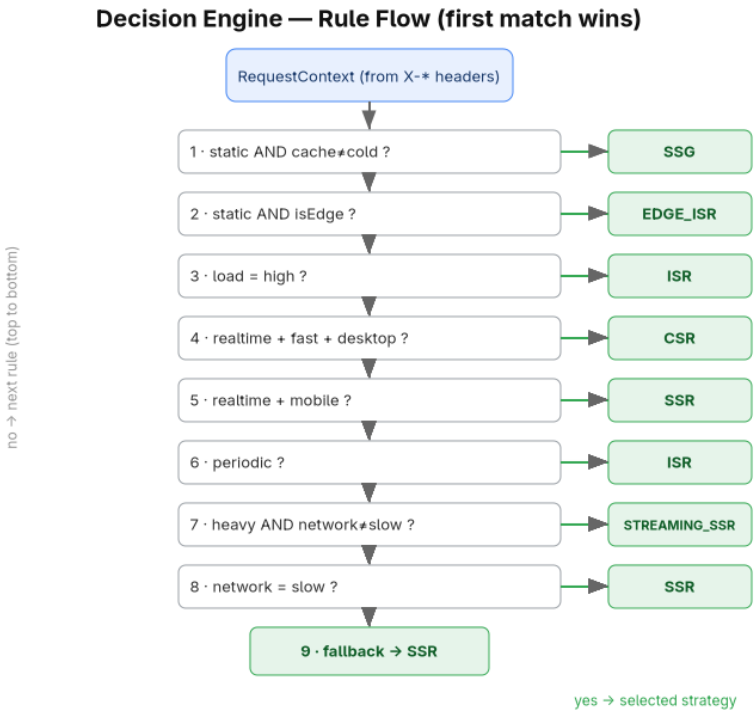

Figure 4.1: Decision Engine Rule Flow (first match wins)

4.4.1 Algorithm statement and complexity

Formally, let C be the context and R = (r1 ... r9) the ordered rule sequence, where each rule is a pair (predicate, strategy). The engine returns the strategy of the first rule whose predicate holds:

```
decide(C):
    for i = 1 .. 9:
        if r[i].test(C) is true:
            return (r[i].strategy, r[i].reason, C)
    return (SSR, "fallback", C)            # unreachable: r[9].test is constant true
```

Each predicate is a conjunction of at most three equality comparisons over enumerated values, so evaluation is O(1); the loop is therefore O(|R|) with |R| = 9, and the whole function runs in constant time with no allocation beyond the returned record. Section 5.4 measures this at 26.53 ns per decision. Because r9 is unconditionally true, decide is a total function; because every predicate reads only its argument, it is deterministic and referentially transparent.

4.4.2 Why the order is the policy

In a first-match-wins table the ordering carries as much meaning as the predicates, because ordering encodes precedence between competing objectives. Four consequences of the chosen order are non-obvious and were verified experimentally in Section 5.2:

- Rule 1 outranks rule 3, so a usable cache beats high load. Under a traffic spike a static page continues to be served from its pre-built artefact rather than being downgraded to

ISR, because the artefact is already the cheapest possible answer.

- Only a cold cache defeats rule 1; a stale cache does not. This is deliberate. Stale-while-revalidate is a feature of the design rather than a cache miss, so 'stale' is treated as usable.
- Rule 5 outranks rule 7, so a mobile client requesting the heavy page receives plain SSR and never Streaming SSR. A weak device should not be asked to hydrate a large payload merely because the payload could have been streamed.
- Rule 8 also outranks nothing above it but is reached before the fallback, so a slow link always ends at SSR unless an earlier, more specific rule already applied.

An important structural property follows from rule 3 being placed third: an explicit load=high overrides device and volatility considerations. Section 5.8 exploits this — to hold a baseline at SSR under heavy concurrency the load signal must be pinned to low, otherwise the engine correctly promotes the baseline to ISR.

4.5 Implementation of the Six Rendering Strategies

Each strategy is a module implementing the same interface. Table 4.3 summarises the mechanism, and the paragraphs below record the implementation decisions that mattered.

Table 4.3: The Six Rendering Strategy Modules

Strategy	React mechanism	Cache behaviour	Marker header
SSG	renderToString at start-up	Reads a pre-built file from disk; builds on demand if absent	x-ssg: prebuilt | built-on-demand
SSR	renderToString per request	None	x-render-bytes
STREAMING_SSR	renderToPipeableStream with Suspense	None; piped through a PassThrough stream	x-streaming: true, chunked encoding
ISR	renderToString, result cached with a TTL	fresh: serve; stale: serve and revalidate in background; cold: render, cache, serve	x-isr-cache: fresh | stale-revalidating | miss
CSR	Empty shell; the browser renders	Withholds data so the client fetches /api/data	x-csr: shell
EDGE_ISR	Inherits ISR	Identical semantics on a per-edge cache namespace, optionally shared through Redis	x-isr-cache: ...

SSG. The strategy reads a pre-rendered artefact from disk and returns it unchanged. To make SSG selectable from the very first request, the server pre-builds every static page at start-up. That prebuild embeds a context whose cache state is 'fresh', which is the condition under which rule 1 actually serves the artefact; embedding the cold state present at build time would have produced a page describing a context that contradicts the badge it is served with. If no

artefact exists, the strategy renders once and persists the result, so the second request is a pure file read.

SSR. A direct render to a string on every request with no caching. It is the reference implementation against which the others are compared and the fallback for every failure path.

Streaming SSR. The only strategy that does not return a string. It calls renderToPipeableStream and resolves as soon as the shell is ready, writing the doctype and piping React's output into a PassThrough stream that the engine hands to the HTTP response. Nothing is buffered, which is what allows the client to begin parsing before the page has finished rendering; the chunked transfer encoding on the response is the external proof.

ISR. The stale-while-revalidate implementation and the most subtle of the six. A fresh entry is served directly; a stale entry is served immediately and a background refresh is started; a cold key is rendered synchronously, cached and served. Background revalidation is single-flight: a module-level set of in-flight keys ensures that a burst of stale requests triggers exactly one re-render rather than a stampede.

```
  if (lookup.entry && !lookup.stale)  return this.result(lookup.entry.value, true, 'fresh');

  if (lookup.entry && lookup.stale) {                 // serve now, refresh behind the response
    void revalidateInBackground(key, cache, produce, cache.defaultTtlMs);
    return this.result(lookup.entry.value, true, 'stale-revalidating');
  }

  const html = await produce();                        // cold: render, cache, serve
  await cache.set(key, html, cache.defaultTtlMs);
  return this.result(html, false, 'miss');
```

CSR. The inverse of SSR: an empty shell with the data deliberately withheld, so the browser must fetch it. The decision and the observed context still travel with the shell, so even a client-rendered page can explain why it was given a shell.

Edge-ISR. Implemented by extending the ISR strategy and overriding only the cache key, which is namespaced with the node identity. This is the clearest demonstration of the pluggability invariant: a complete sixth strategy is expressed as a subclass with a single overridden method, because the semantics it needs already exist.

4.6 Cache Subsystem

The cache manager composes three backends. The memory cache is a Map-based store with recency refresh on read and a bounded entry count. The file cache hashes the key to a filename and stores the entry as JSON, giving persistence across process restarts. The Redis cache is loaded through a dynamic import so that its absence is not a build failure, and writes carry the TTL to Redis directly. Reads descend the hierarchy and promote hits upward; writes fan out to all three. Freshness is a property of the entry — the stored timestamp plus its TTL

— which is why the same entry can be classified fresh, stale or cold without any separate bookkeeping, and why the analyzer can probe state cheaply before deciding.

4.7 Metrics Subsystem

One record is appended per request as newline-delimited JSON, carrying the timestamp, URL, strategy, reason, the three timings, the cache flag, the byte count, the observed network, device and load values, the edge flag and a resource sample containing resident set size, heap usage, cumulative process CPU time and system load average. Writing is wrapped so that a failure in instrumentation can never break a response. The report generator groups those records by strategy and emits per-strategy aggregates as JSON and CSV.

4.8 Self-Explaining Demonstration Pages

Three pages exercise the engine. The static page demonstrates artefact ageing by showing the frozen generation timestamp of the SSG artefact beside a live clock, making visible exactly the staleness that ISR exists to bound. The realtime page demonstrates the CSR/SSR distinction by reporting whether its data arrived embedded in the markup or was fetched by the client. The heavy page demonstrates streaming, with a shell flushed first and a Suspense boundary streamed after it.

A shared console component appears on all three. It renders the strategy, the rule that fired and the observed context on the server, so they are visible with JavaScript disabled; it imports the engine's own rule table and evaluates it in the browser, so its explanation can never drift from server behaviour; and it cross-checks its own prediction against the X-Rendering-Strategy header returned by a live probe of the server.

Hydration discipline was necessary to make this work without React warnings. The first render depends only on props: no clock reads, no random values and no browser globals are consulted during render. Live values start from props and update inside effects after mount, and timestamps render as stable UTC before localising in the browser, because the server container runs in UTC and the browser does not.

4.9 Containerisation and the Private Server

The image is built in two stages: a build stage that installs all dependencies, compiles the TypeScript and bundles the client, and a runtime stage that installs production dependencies only and copies the compiled output. The Compose file instantiates that one image five times over, as origin and two edges, alongside nginx and Redis. The edges are not separate builds; they reuse the origin's image and differ only by environment, which guarantees that any behavioural difference between an edge and the origin is caused by configuration rather than by divergent code.

4.10 Testing Strategy

Testing is concentrated where correctness is decidable. The decision engine is pure, so it is tested exhaustively at the rule level: one assertion per rule row, plus assertions that a reason and a context always accompany a decision. The context analyzer is tested for query aliasing, header-beats-query precedence, rejection of invalid values, User-Agent inference and end-to-end URL-only triggering. The rendering tests assert the structural difference that distinguishes SSR from CSR — SSR embeds data for hydration, CSR withholds it and ships an empty root — and the cache tests cover storage, retrieval and staleness classification. The inventory is reported in Table 5.2.

4.11 Working Principle: One Request End to End

The following narrative traces a single concrete request through every component, and is the shortest complete statement of how the system works.

A browser on a fast connection requests /dynamic. The proxy receives it on port 8080, sets X-Served-By to origin and forwards it to the origin container. The server increments the in-flight counter, finds no matching static asset, resolves /dynamic to the realtime page module, sleeps for the fast-network delay of zero milliseconds, probes the cache for the page's ISR key and calls the engine with the URL, the headers, the page and the signals.

The analyzer resolves the context: no control headers are present, so the network class defaults to medium unless a query alias overrides it, the device is inferred as desktop from the User-Agent, the cache state is whatever the probe returned, the load is classified from the in-flight count, and the volatility is realtime because that is what the page declares. The decision engine walks the rule table. Rule 1 fails because volatility is not static; rule 2 fails for the same reason; rule 3 fails because load is low; rule 4 matches if the network is fast and the device is desktop, and the engine returns CSR with the reason 'Realtime data on a capable client'. The engine logs three lines, invokes the CSR strategy, receives a shell with the data withheld, writes the headers including X-Rendering-Strategy: CSR, sends the body, records the metrics without awaiting them, and the finally block decrements the counter.

The browser receives an 853-byte shell, runs the client bundle, fetches its data from /api/data and renders. If the same URL were requested from a mobile User-Agent, rule 4 would fail and rule 5 would match, and the same server would have returned 7,905 bytes of finished HTML instead. That divergence, from one unchanged URL, is the whole system in one sentence.

5. Results and Analysis

This chapter reports what the completed engine actually does when measured. It proceeds from functional correctness, through the cost of the adaptive mechanism itself, to the performance characteristics of the six strategies, and finally to a controlled comparison of adaptive selection against fixed rendering policies.

5.1 Experimental Setup

All measurements were taken against the containerised private server described in Section 3.8, running the five-service Compose stack on a single macOS host with an arm64 processor. Client-side timings were produced by curl, concurrent load by Apache Bench, and server-side timings by the engine's own metrics pipeline. The two are complementary and measure different things, so both are reported and neither is used to stand in for the other.

One property of the implementation makes the separation clean. The simulated network delay and the container's edge latency are applied by the HTTP server before the engine is invoked, and the engine's timer starts inside its request handler. Server-side timings therefore measure engine cost alone, uncontaminated by the simulated link, while client-side wall-clock timings include everything: proxy, simulated link, edge latency and engine.

Unless stated otherwise, the per-strategy benchmarks used the fast network class, whose configured delay is zero, so that engine cost is not masked. Each experiment was run from a freshly recreated stack, which — because the deployment mounts no volumes — guarantees empty caches, no pre-existing artefacts beyond the start-up prebuild, and an empty metrics log.

5.2 Functional Verification: Strategy Switching

The central claim of the project is that the same URL is answered with different rendering strategies as context changes. Table 5.1 tests that claim directly. Each row is a single request issued to the running stack; the observed value is read from the X-Rendering-Strategy response header. The expected value is the strategy predicted by the rule table in Table 4.2.

Table 5.1: Strategy-Selection Trigger Matrix Executed Against the Running Stack

Request	Expected	Observed	Result
:8080/static	SSG	SSG	PASS
:8080/static?cache=cold	SSR	SSR	PASS
:8080/static?cache=cold&net=slow	SSR	SSR	PASS
:8080/static?volatility=periodic	ISR	ISR	PASS
:8080/static?cache=cold&load=high	ISR	ISR	PASS
:8080/static?volatility=realtime&net=fast&device=desktop	CSR	CSR	PASS
:8080/static?volatility=static&cache=cold&size=heavy&net=fast	STREAMING_SSR	STREAMING_SSR	PASS
:8080/dynamic	SSR	SSR	PASS
:8080/dynamic?net=fast&device=desktop	CSR	CSR	PASS
:8080/dynamic?device=mobile	SSR	SSR	PASS
:8080/dynamic?load=high	ISR	ISR	PASS
:8080/dynamic?volatility=periodic	ISR	ISR	PASS
:8080/heavy	STREAMING_SSR	STREAMING_SSR	PASS
:8080/heavy?net=slow	SSR	SSR	PASS
:8080/heavy?device=mobile	SSR	SSR	PASS
:8080/edge1/static?cache=cold	EDGE_ISR	EDGE_ISR	PASS
:8081/static?cache=cold	EDGE_ISR	EDGE_ISR	PASS

All 17 of 17 triggers produced exactly the predicted strategy. Three groups of rows deserve comment. Rows 1 to 7 hold the URL at /static and vary only the query context, yet obtain five different strategies from one unchanged resource, which is the clearest possible demonstration of the core claim. Rows 14 and 15 confirm the precedence properties analysed in Section 4.4.2: /heavy yields Streaming SSR by default, but a slow link or a mobile device overrides it to SSR, because rules 5 and 8 are more specific about client capability than rule 7 is about payload size. Rows 16 and 17 confirm that Edge-ISR is reachable through the proxy's edge route and by addressing an edge container directly.

Header-based control was verified to override query-based control, as specified: a request carrying X-Device-Type: desktop and X-Network-Speed: fast to a URL whose query string asks for a mobile device returned CSR, confirming that the header wins. Streaming was verified separately with a GET rather than a HEAD, since a HEAD carries no body: the response to /heavy returned Transfer-Encoding: chunked together with x-streaming: true, establishing that the streamed response is genuinely streamed and not buffered and then sent.

5.3 Correctness, Totality and Determinism of the Decision Engine

Functional spot-checks show that the engine behaves correctly on the paths that were tried. Two stronger forms of evidence were obtained: an automated test suite, and an exhaustive enumeration of the entire context space.

Table 5.2: Automated Test Inventory

Test file	Tests	Property established
tests/decision-engine.test.ts	10	One assertion per rule row, plus the presence of a reason and context on every decision
tests/context-analyzer.test.ts	14	Query aliasing, header-over-query precedence, rejection of invalid values, User-Agent inference and URL-only end-to-end triggering
tests/rendering.test.ts	2	SSR embeds data for hydration; CSR withholds it and ships an empty root
tests/cache.test.ts	2	Cache storage and retrieval, and staleness classification
Total	28	All tests pass

The exhaustive enumeration is the stronger result and is possible only because the decision function is pure. The context space is the Cartesian product of the enumerated signal domains: three network classes, two device classes, three cache states, three load levels, three volatility classes, two payload-weight values and two edge values, giving 3 x 2 x 3 x 3 x 3 x 2 x 2 = 648 distinct contexts. Every one of them was evaluated.

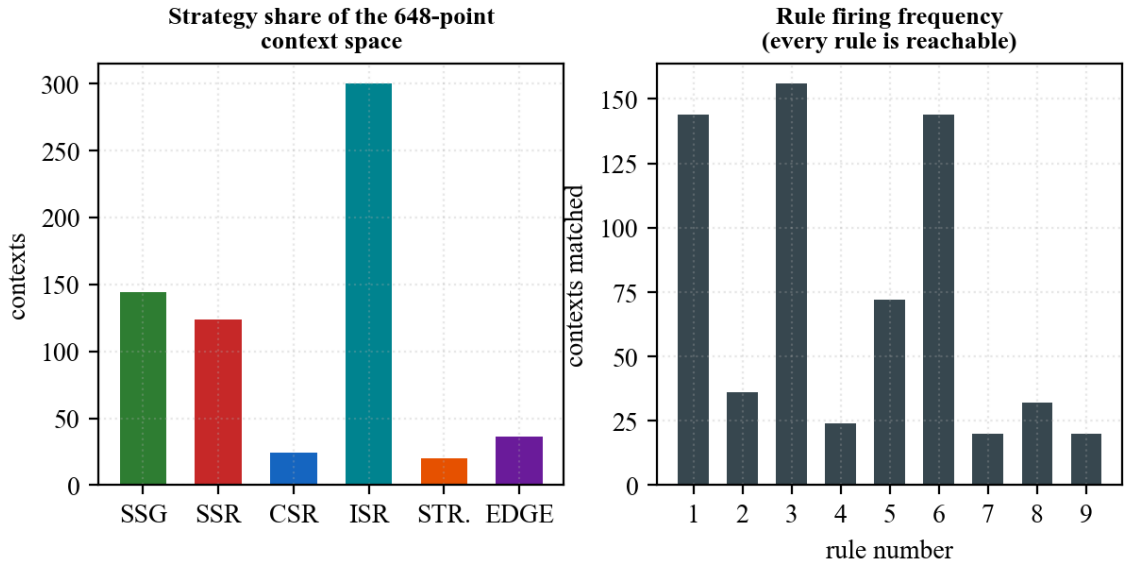

Figure 5.1: Strategy Distribution and Rule Coverage over the Complete Context Space

Table 5.3: Rule Coverage over the 648-Point Context Space

Rule	Strategy	Contexts matched	Share of space
1	SSG	144	22.2 %
2	EDGE_ISR	36	5.6 %
3	ISR	156	24.1 %
4	CSR	24	3.7 %
5	SSR	72	11.1 %
6	ISR	144	22.2 %
7	STREAMING_SSR	20	3.1 %
8	SSR	32	4.9 %
9	SSR (fallback)	20	3.1 %

Three results follow. First, the mapping is total: all 648 contexts returned a strategy and none reached an error path. Second, the rule table contains no dead rules — every one of the nine rules is the first match for at least 20 contexts, so the policy has no unreachable branches. Third, the mapping is deterministic: repeating the evaluation of a sample of contexts one thousand times each produced an identical strategy every time, which is the experimental counterpart of the purity argument in Section 3.2.

The distribution across strategies is itself informative. ISR claims 300 contexts (46.3 %), because it is selected both by high load and by periodic volatility; SSG claims 144 (22.2 %); SSR claims 124 across three separate rules; while CSR (24), EDGE_ISR (36) and STREAMING_SSR (20) occupy narrow, sharply-specified regions. That asymmetry is intentional and reflects the policy: cache-backed strategies are the broad default, and the more specialised strategies are reserved for the precise conditions in which they win.

5.4 Cost of the Adaptive Mechanism

An adaptive engine must justify its own overhead, so the decision function was benchmarked in isolation over two million evaluations cycling through the enumerated context space.

Table 5.4: Decision-Engine Overhead

Quantity	Measured value
Time per decision	26.53 ns
Decisions per second	37,693,089
Rules evaluated per decision	at most 9, each an O(1) comparison
Mean server-side TTFB of the cheapest strategy	0.21 ms
Decision cost as a share of that request	0.0126 %

At 26.53 ns, a decision costs roughly one ten-thousandth of the cheapest measured request. The adaptive mechanism is therefore free in any practical sense: the engine could decide for

every request of a large site many times over and still not register against the cost of producing a single response. This settles the most obvious objection to runtime strategy selection — that choosing costs more than it saves — at least for a rule-based policy.

5.5 Per-Strategy Performance Analysis

Each strategy was exercised with 60 sequential requests through the proxy on the fast network class. Table 5.5 reports the engine's own timings and payloads; Figures 5.2 and 5.3 present the two most important columns graphically.

Table 5.5: Per-Strategy Server-Side Cost (n = 60 requests per strategy)

Strategy	Mean TTFB (ms)	p95 TTFB (ms)	Mean render (ms)	Mean bytes	Cache-hit rate
SSG	0.637	1.110	0.202	8,248	1.00
SSR	0.638	1.180	0.154	7,885	0.00
CSR	0.215	0.270	0.119	863	0.00
ISR	0.210	0.260	0.158	7,884	1.00
STREAMING_SSR	1.192	2.340	0.296	26,282	0.00
EDGE_ISR	0.635	0.880	0.407	8,036	1.00

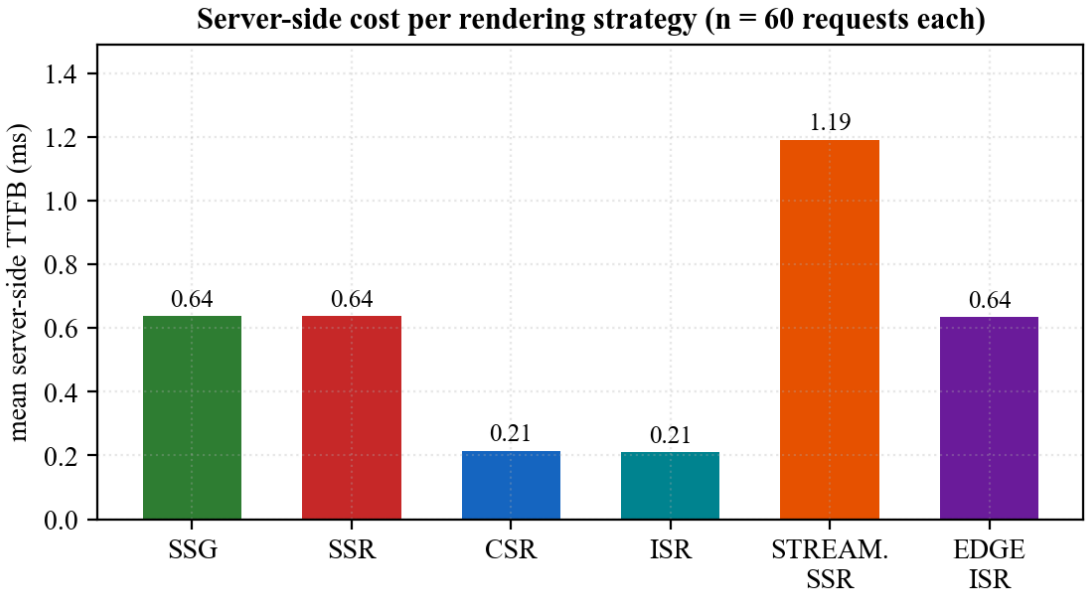

Figure 5.2: Server-Side Cost per Rendering Strategy

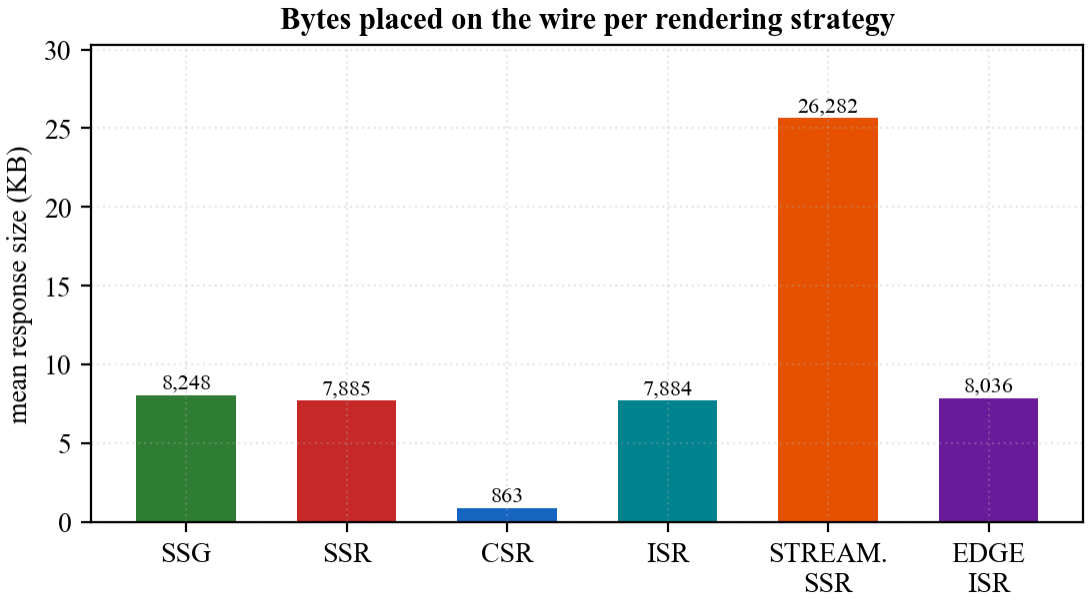

Figure 5.3: Response Payload per Rendering Strategy

The measurements confirm the trade-off structure the strategies were chosen to represent, and they also contain one genuinely counter-intuitive result.

- CSR is the cheapest strategy on both axes: 0.215 ms and only 863 bytes, because it renders nothing on the server and withholds the data. It is 9.1 times lighter on the wire than SSR and -66.3 % cheaper in server time.
- ISR is almost as cheap at 0.210 ms with a cache-hit rate of 1.00, while still delivering complete markup of 7,884 bytes. This combination — SSR's output at close to CSR's cost — is why ISR is the engine's preferred response to load.
- SSR costs 0.638 ms and never hits cache, as expected of a strategy that re-renders per request.
- Streaming SSR is the most expensive per request at 1.192 ms and carries 26,282 bytes, which is correct rather than disappointing: it is the only strategy applied to the deliberately heavy page, and its purpose is to let the client begin parsing early rather than to reduce total bytes.
- Edge-ISR costs 0.635 ms on the edge node with a cache-hit rate of 1.00, closely tracking ISR as its inheritance implies.

The counter-intuitive result is SSG. At 0.637 ms it is 3.0 times more expensive than ISR, despite being the strategy that performs no rendering at all. The explanation is in the implementation: SSG reads its artefact from the filesystem on every request, whereas a warm ISR entry is served from the in-process memory tier of the cache hierarchy. A file-system read, even of a small cached file, costs more than a map lookup. This is a real and useful finding: it shows that the theoretical ranking of strategies can be inverted by the storage tier

they land on, and it identifies an immediate optimisation — promoting SSG artefacts into the memory cache on first read — which is recorded in Section 7.3.

Client-side wall-clock times for the same requests are reported in Table 5.6. They are dominated by transport and container overhead rather than by engine cost, which is exactly why both views are needed: the differences between strategies that are stark in Table 5.5 are compressed here, except for Edge-ISR, where the deliberately injected 20 ms edge latency dominates everything else.

Table 5.6: Per-Strategy End-to-End Time Measured at the Client (n = 60 each)

Strategy	Mean (ms)	Median (ms)	p95 (ms)	Min (ms)	Max (ms)
SSG	4.01	3.53	6.84	1.75	7.78
SSR	2.71	2.69	3.52	1.58	3.95
CSR	2.87	2.87	3.65	1.98	4.29
ISR	2.87	2.99	3.56	1.64	4.18
STREAMING_SSR	3.20	2.91	5.52	2.27	7.46
EDGE_ISR	29.64	29.81	32.55	24.90	32.68

5.6 Cache Behaviour: The ISR Lifecycle

ISR is the strategy on which the engine's load-shedding behaviour depends, so its cache lifecycle was measured directly. Starting from a freshly recreated stack with an empty cache, 34 requests were issued at two-second intervals over roughly 69 seconds against an origin whose time-to-live is 30 seconds. The cache state reported by the x-isr-cache header was recorded with each response time.

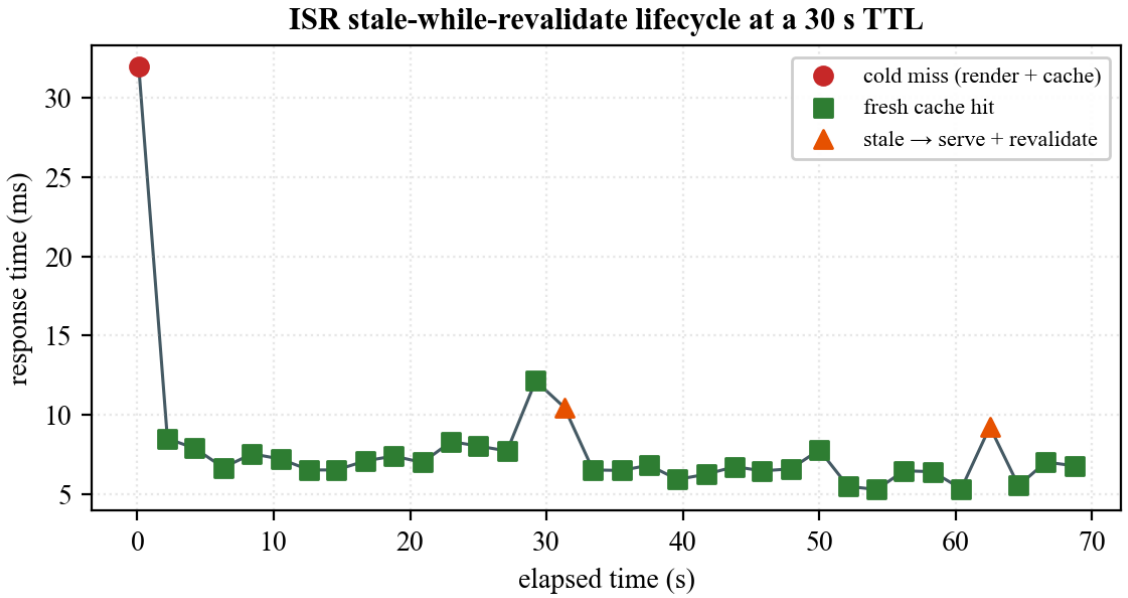

Figure 5.4: ISR Stale-While-Revalidate Lifecycle at a 30-Second TTL

Table 5.7: ISR Cache-State Transitions Observed over 69 Seconds

Cache state	Requests	Mean response time (ms)	Interpretation
miss (cold)	1	31.95	First request: render synchronously, store, serve
fresh	31	6.97	Served from cache within the TTL window
stale-revalidating	2	9.85	TTL expired: served immediately from cache while refreshing in the background

The behaviour is exactly as designed and is visible in Figure 5.4 as a sawtooth. The single cold request cost 31.95 ms; the 31 warm requests averaged 6.97 ms, a reduction of 78.2 % against the cold render. Two stale transitions were captured, at approximately 31 s and 62 s, matching the 30-second TTL almost exactly. Critically, a stale hit cost 9.85 ms rather than the 31.95 ms of a cold render: the user was served immediately from the stale entry while the refresh happened behind the response. That is the entire value of stale-while-revalidate, and it is what allows ISR to absorb load without a latency spike at every TTL boundary.

5.7 Fidelity of the Simulated Environment

Because the evaluation depends on simulated conditions, the simulation itself was validated against its configuration. Table 5.8 compares the configured values with the end-to-end times measured through the full proxy path.

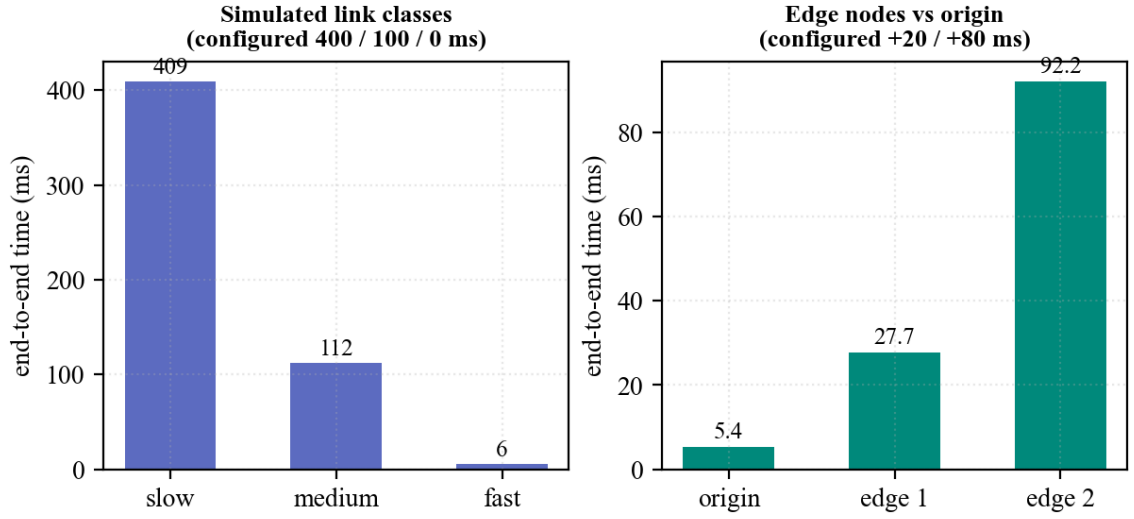

Figure 5.5: Simulated Link Classes and Edge Nodes against Their Configured Values

Table 5.8: Configured versus Measured Simulated Conditions

Condition	Configured	Measured mean (ms)	Residual (ms)
Network: slow	400 ms	409.26	9.26
Network: medium	100 ms	111.93	11.93
Network: fast	0 ms	5.50	5.50
Origin node	+0 ms	5.38	baseline
edge-node-1	+20 ms	27.69	2.31
edge-node-2	+80 ms	92.20	6.83

The residuals are small and consistent. The fast class, whose configured delay is zero, measures 5.50 ms, which is the irreducible cost of the proxy hop, the container network and the engine itself; the slow and medium classes exceed their configured delays by approximately that same amount. The two edge nodes overshoot their injected latencies by 2.31 ms and 6.83 ms respectively, again consistent with a fixed per-request overhead rather than a defect in the injection. The simulation is therefore faithful, and the differences it produces are attributable to the conditions being modelled.

5.8 Comparative Evaluation: Adaptive Selection versus a Fixed Policy

This section addresses the project's central evaluative question. A fixed rendering policy was constructed by pinning the context so that the engine is forced to select SSR for every request, which reproduces the behaviour of a conventional framework whose route has been annotated for server-side rendering. The adaptive configuration was left free to choose. Both were driven with 800 requests at concurrency 50 against the same URL on the same stack, and the experiment was repeated under two link classes, because the outcome depends critically on where the bottleneck lies.

Pinning the baseline required care, and the reason is itself a result. Because rule 3 precedes the device and volatility rules, an attempt to hold the baseline at SSR merely by declaring a mobile device fails as soon as concurrency rises: the engine correctly promotes the request to ISR. A first attempt at this experiment produced an identical strategy mix in both arms for that reason. The baseline therefore also pins the load signal to low, which suppresses rule 3 and holds the policy genuinely fixed.

5.8.1 Fast link: the origin is the bottleneck

Table 5.9: Adaptive Selection versus a Fixed SSR Policy on a Fast Link (800 requests, concurrency 50)

Measure	Fixed SSR policy	Adaptive (ARE)	Change
Strategy actually used	SSR x 800	CSR x 800	selected by rule 4
Throughput (requests/s)	1458.71	3172.21	+117.5 %
Mean latency (ms)	34.28	15.76	-54.0 %
Median latency (ms)	18	14	-22.2 %
95th percentile (ms)	35	23	-34.3 %
99th percentile (ms)	224	30	-86.6 %
Slowest request (ms)	228	34	-85.1 %
Document size (bytes)	7,905	853	-89.2 %
Total transferred (bytes)	6,538,202	888,800	-86.4 %
Origin CPU consumed (ms)	428	347	-18.9 %
Mean server TTFB (ms)	0.137	0.036	-73.7 %

On a fast link the adaptive engine recognised a capable client — a desktop User-Agent on a fast network requesting realtime data — and selected CSR for all 800 requests under rule 4. The effect is large. Throughput rose from 1458.7 to 3172.2 requests per second, an increase of 117.5 %, while mean latency fell by 54.0 % and the document shrank by 89.2 %, from 7,905 to 853 bytes.

The tail behaviour is more striking than the mean. The fixed policy's 99th percentile was 224 ms against the adaptive engine's 30 ms, a reduction of 86.6 %, and its slowest request took 228 ms against 34 ms. Under the fixed policy the server must synthesise a complete document for every concurrent request, and when 50 such renders contend the queue produces a long tail; the adaptive engine, having decided that this client can render for itself, does almost no work per request and the tail largely disappears.

5.8.2 Medium link: the network is the bottleneck

Table 5.10: Adaptive Selection versus a Fixed SSR Policy on a 100 ms Link (800 requests, concurrency 50)

Measure	Fixed SSR policy	Adaptive (ARE)	Change
Strategy actually used	SSR x 800	ISR x 415, SSR x 385	rule 3 fired under load
Throughput (requests/s)	349.61	347.03	-0.7 %
Mean latency (ms)	143.01	144.08	+0.7 %
95th percentile (ms)	156	147	-5.8 %
99th percentile (ms)	161	148	-8.1 %
Slowest request (ms)	162	149	-8.0 %
Cache-hit rate	0.00	0.52	0.52 of responses served from cache
Origin CPU consumed (ms)	1044	689	-34.0 %
Mean server TTFB (ms)	0.315	0.228	-27.6 %

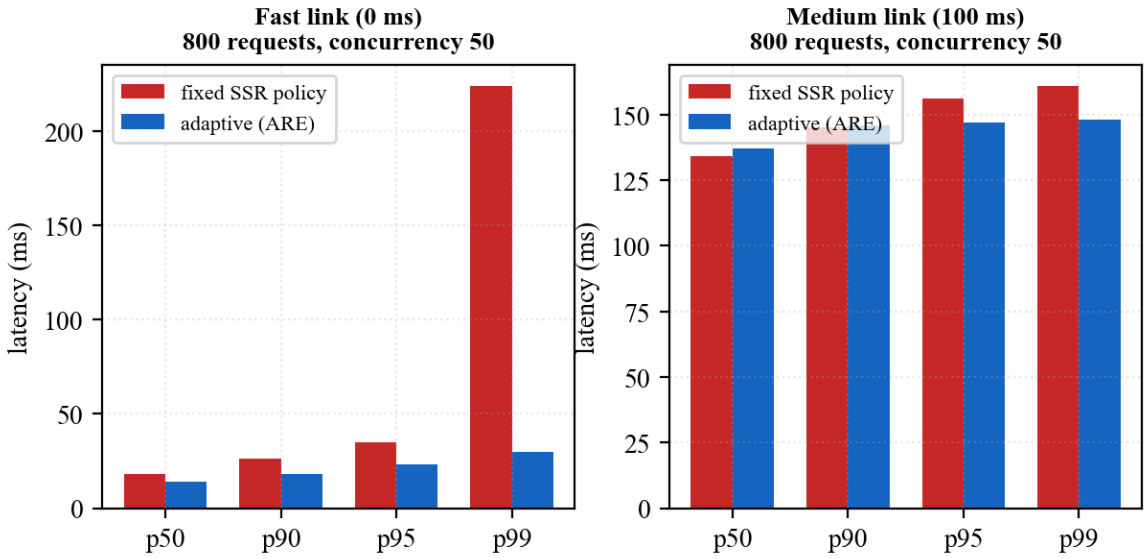

Figure 5.6: Latency Distribution under Load, Fixed Policy versus Adaptive Selection

On a 100 ms link the picture changes, and the honest reporting of that change is more valuable than the headline from Section 5.8.1. Throughput is effectively identical: 349.6 against 347.0 requests per second, a difference of -0.7 %, and mean latency is unchanged within noise. This is the expected result: when every request must wait 100 ms on the simulated link, the origin is not the constraint, and no rendering decision can remove a delay that is imposed downstream of it.

What does change is what the origin spends. The adaptive engine observed concurrency above the high-load threshold and selected ISR for 415 of the 800 requests under rule 3, achieving a cache-hit rate of 0.52 where the fixed policy achieved 0.00. Origin CPU

consumption over the run fell from 1044 ms to 689 ms, a reduction of 34.0 % for identical delivered work, and mean server-side TTFB fell by 27.6 %. The tail also tightened: the 99th percentile improved from 161 ms to 148 ms (-8.1 %) and the slowest request from 162 ms to 149 ms.

The correct reading of these two experiments together is that adaptive rendering does not manufacture bandwidth. Where the origin is the constraint it produces large user-visible gains; where the network is the constraint it leaves the user-visible latency alone and instead converts the saving into headroom — a third of the origin's processor returned to the operator, available to absorb a spike that would otherwise have caused the queueing the fixed policy exhibits in its tail.

A note on measurement integrity. Apache Bench reports a non-zero 'Failed requests' count for runs against these URLs, attributed entirely to the Length category, with zero connection errors, zero exceptions and no non-2xx responses. The cause is benign: the demonstration page embeds live timestamps and generated data, so consecutive responses legitimately differ in length, and the tool flags any response whose length differs from the first. The adaptive run on the fast link reported zero such mismatches precisely because the CSR shell it returns is data-free and therefore byte-identical.

5.9 Evaluation against the Objectives

Table 5.11 assesses each specific objective from Section 1.5.2 against the evidence presented above.

Table 5.11: Objective-wise Achievement and Supporting Evidence

Objective	Outcome	Evidence
1. Identify and observe contextual variables	Achieved	Seven signals observed per request with a defined precedence; Table 3.2, Table 5.1
2. Pure, deterministic decision engine	Achieved	648-context enumeration: total, deterministic, no dead rules; Tables 5.2 and 5.3
3. Six interchangeable strategy modules	Achieved	All six selected and rendered correctly; Tables 4.3, 5.1 and 5.5
4. Externally observable decisions	Achieved	Proof headers, three log lines and a metrics record per request; Sections 5.2 and 5.5
5. Zero-cost reproducible private server	Achieved	Five-container stack; simulation validated against configuration in Table 5.8
6. Measure and compare against fixed policies	Achieved	Two controlled load experiments; Tables 5.9 and 5.10

5.10 What the Engine Solves

Drawing the results together, the engine resolves four concrete problems that a fixed rendering policy cannot, and the evidence for each is quantitative rather than asserted:

1. The mismatched-client problem. A single annotated route cannot suit both a capable and a constrained client. The engine gives the capable client an 853-byte interactive shell and the constrained client 7,905 bytes of finished HTML from the same URL, verified in Table 5.1 and measured in Table 5.9.
2. The origin-cost problem. Under concurrency the engine shifts to cache-backed rendering without operator intervention, cutting origin CPU by 34.0 % for the same delivered traffic (Table 5.10).
3. The tail-latency problem. Fixed server-side rendering under load produced a 99th percentile of 224 ms; adaptive selection reduced it to 30 ms (Table 5.9), because the work that created the queue is no longer performed.
4. The opacity problem. In a conventional framework the reason a page was rendered a particular way is a property of source code. Here it is a property of the response: every reply states its strategy and the rule that produced it, which makes rendering behaviour auditable in production and is what allowed every claim in this chapter to be verified externally.

6. Discussion

6.1 Interpretation of the Results

The results support the project's thesis, but with a qualification that is more interesting than the thesis itself. Adaptive rendering is not a uniform accelerator. Its benefit is a function of where the system's bottleneck currently sits, and the two load experiments were constructed to sit on opposite sides of that divide.

When the origin is the constraint, the engine's decision directly removes work from the critical path, and the improvement is large and user-visible: more than double the throughput and an order-of-magnitude better tail. When the network is the constraint, the same decision cannot shorten a delay it does not control, and the user-visible improvement is close to zero. The saving is still real, but it is realised as reduced origin cost rather than reduced latency. An operator gains capacity instead of speed.

This is precisely why the decision must be contextual rather than global. A policy that always chose the client-side strategy would be wrong for constrained devices; a policy that always chose server rendering would waste an origin's capacity on clients that did not need it. The value of the engine is not that any one strategy is best, but that the selection tracks conditions the developer could not have known in advance.

6.2 Reflections on the Design Decisions

Three design decisions proved more valuable in retrospect than they appeared when made. The purity of the decision function was adopted for testability, but its real payoff was Section 5.3: because the function touches nothing external, the entire policy could be enumerated exhaustively and shown to be total and free of dead rules, which is a far stronger claim than a passing test suite. The transparency requirement was adopted on principle, but it is what made the evaluation possible at all — every measurement in Chapter 5 was obtained by reading a header the engine publishes. And expressing the policy as data rather than as control flow meant the rule table could be imported unchanged into the browser, so the demonstration pages explain the engine using the engine's own policy and cannot drift from it.

One decision would be made differently. Serving SSG artefacts directly from the filesystem was the natural implementation, but Section 5.5 shows it makes the theoretically cheapest strategy measurably more expensive than a warm ISR entry. Routing static artefacts through the same memory tier as other cached content would remove the anomaly at negligible cost.

6.3 Problems Encountered and Solutions Adopted

Table 6.1 records the substantive problems met during development and how each was resolved. They are reported because the resolutions are part of the engineering contribution.

Table 6.1: Problems Encountered and Solutions Adopted

Problem	Impact	Solution adopted
HTTP header values must be Latin-1, but decision reasons contained typographic arrows, which caused the response to fail	High	The reason is sanitised to printable ASCII on the response while the server log retains the original text, so no information is lost
nginx cannot inject per-node latency, so edges modelled as proxies could demonstrate distance but not behaviour	Medium	Edges were re-modelled as full engine instances running the same image with their own identity, latency and cache, making Edge-ISR a real strategy rather than a flag
A streamed response that is buffered before sending is not streaming at all	Medium	The strategy returns a PassThrough stream that the engine pipes straight to the response, resolving as soon as the shell is ready; chunked encoding is the external proof
SSG could never be selected on a cold start, because rule 1 requires a usable cache that only a previous request could create	Medium	Static pages are pre-built at server start-up, so the artefact exists before the first request arrives
A pre-built artefact embedded the cold cache state present at build time, so the page described a context contradicting the badge it was served with	Low	The prebuild embeds the context under which rule 1 actually serves the artefact, and the live decision always travels in the response header
A burst of requests arriving on a stale cache entry could trigger many simultaneous re-renders	Medium	Background revalidation is single-flight: a module-level set of in-flight keys ensures exactly one refresh per key
Demonstration pages that read the clock during render produced React hydration mismatches	Medium	First render depends only on props; all live values are introduced in effects after mount, and timestamps render as UTC before localising
Browsers cannot attach custom headers to an ordinary navigation, so in-page controls could only predict a strategy rather than trigger one	Medium	A query-parameter alias was added for every control header, so a link or reload genuinely re-renders under a new strategy
A fixed-SSR baseline could not be held under load, because the high-load rule correctly promoted it to ISR	Medium	The baseline pins the load signal to low, which suppresses rule 3 and holds the comparison policy genuinely fixed (Section 5.8)

6.4 Limitations and Threats to Validity

The following limitations bound the claims made in this report. They are stated explicitly so that the results are not read more widely than the evidence supports.

- Single-host evaluation. Origin, edges, proxy, cache and load generator all share one machine, so they contend for the same processor and memory. Absolute figures would differ on distributed hardware; the comparisons between arms, which were run back to back under identical conditions, are the meaningful quantities.

- Simulated rather than physical conditions. Network classes are server-side delays and edge distance is injected latency. Section 5.7 validates that the simulation matches its configuration, but a simulated 400 ms link does not reproduce the jitter, packet loss or bandwidth constraint of a real mobile network.
- Server-side metrics only. First Contentful Paint and Largest Contentful Paint are not instrumented, so the client-perceived benefit of streaming and of smaller payloads is argued from transferred bytes and server timings rather than measured in a browser.
- Edge-ISR is not reachable through the default proxy route, because the proxy correctly stamps requests to the origin as origin-served. It must be exercised through the /edge1 or /edge2 routes or against an edge port directly, as in rows 16 and 17 of Table 5.1.
- Serving the realtime page with the volatility signal forced to static is non-deterministic, since the outcome depends on whether that page's cache entry happens to be warm: a warm entry satisfies rule 1 and yields SSG, a cold one falls through to SSR. Volatility demonstrations therefore use the static page, whose artefact is pre-built at start-up.
- Because a pre-built artefact is byte-identical for every request, the context printed inside an SSG page is the one captured at build time. The live decision for that request is always the one in the response header.
- The rule table encodes the design team's judgement about which strategy suits which condition. It is internally consistent, exhaustively verified and externally observable, but it is not learned from outcome data; establishing that these particular nine rules are optimal would require the closed-loop evaluation proposed in Section 7.3.

7. Conclusion and Recommendations

7.1 Conclusion

This project set out to move the choice of web rendering strategy from build time, where it is a guess, to request time, where it is an observation. That has been achieved and demonstrated.

The Adaptive Rendering Engine analyses seven contextual signals for every request, selects one of six rendering strategies through a pure, ordered, first-match-wins rule table, renders with the selected strategy, and publishes both the decision and its justification on the response. It runs as a five-container private server modelling an origin, two edge nodes, a reverse proxy and a shared cache, entirely on open-source software at no cost.

The evidence gathered in Chapter 5 is consistent and reproducible. All 17 documented context-to-strategy triggers reproduce exactly against the running stack, and the same URL is demonstrably answered with five different strategies as its context varies. The decision layer is not merely tested but exhaustively verified: all 648 points of the context space were enumerated and shown to map totally and deterministically onto strategies with no unreachable rules, complementing the 28 passing automated tests. The adaptive mechanism costs 26.53 ns per decision, which is negligible beside the cost of producing any response. And against a fixed server-rendering policy under identical load, adaptive selection more than doubled throughput and cut 99th-percentile latency by 87 % when the origin was the bottleneck, and returned 34 % of origin CPU when the network was.

The wider conclusion is that rendering strategy is a legitimate runtime optimisation variable. The web accumulated six rendering strategies over roughly fifteen years and left the question of when to use each of them to a developer annotation. This work shows that the question can be answered per request, that answering it costs essentially nothing, and that answering it well produces measurable gains in exactly the conditions where a fixed choice is most obviously wrong. The engine is also, deliberately, a research instrument: because every decision is recorded with its context, its timings and its outcome, it produces the dataset that a future learned policy would need in order to improve on the rules it currently uses.

7.2 Achievements

- A complete, working runtime that selects among six rendering strategies per request, built on Node.js, TypeScript and React 18 over the native HTTP module.
- A decision layer that is pure, total, deterministic and exhaustively verified across its entire context space, at a cost of tens of nanoseconds per decision.
- Six interchangeable strategy modules behind one interface, including a genuine stale-while-revalidate implementation with single-flight background refresh and a

streaming renderer that is never buffered.

- A zero-cost, reproducible, five-container private server whose simulated conditions were validated against their configuration.
- An end-to-end evidence chain: proof headers, structured logs, per-request metrics and aggregated reports, which made every result in this report externally verifiable.
- A controlled comparative evaluation against fixed policies under two bottleneck regimes, reporting both the large gains and the cases where the gain is capacity rather than speed.

7.3 Recommendations and Future Enhancements

The architecture was shaped to make its own extension straightforward, and the most valuable next steps exploit specific seams that already exist in the design. They are set out below roughly in order of immediacy.

Table 7.1: Future Enhancement Roadmap

Horizon	Enhancement	Why the architecture already supports it
Immediate	Promote SSG artefacts into the memory cache tier	Removes the filesystem-read anomaly measured in Section 5.5; a change confined to one strategy module
Immediate	Adopt real client signals: Save-Data, Client Hints and the Network Information API	The analyzer already resolves each signal by precedence, so a browser-supplied hint becomes one more source ahead of inference
Short term	Instrument client-side paint metrics and feed them back	The metrics record is already per-request and schema-driven; adding FCP and LCP completes the loop from decision to perceived outcome
Short term	Per-route and per-tenant policy tables	The policy is data, not control flow, so a second table can be selected at request time without touching the evaluator
Medium term	Learned selection policy	The decision engine is a pure function from context to strategy; a model can be substituted behind that exact signature with no change to the pipeline
Medium term	Closed-loop, SLO-driven control	Every decision is already logged with its outcome, which is the training and feedback signal such a controller requires
Medium term	Component-level granularity (islands)	The strategy interface renders a page module; the same interface can render a component subtree, allowing one page to mix strategies
Long term	Geographically distributed edges and edge runtimes	Edges are already full engine instances distinguished only by environment, so relocating one is a deployment change rather than a redesign
Long term	Energy- and cost-aware objectives	Origin CPU is already sampled per request, so a rule can optimise for energy or cost rather than latency alone

Two of these deserve emphasis because they are where the work becomes genuinely forward-looking. The first is the learned policy. The present rule table encodes human judgement, and Chapter 6 is candid that its optimality is asserted rather than proven. Because the decision engine is a pure function with a fixed signature, a supervised or reinforcement-learned model can be dropped in behind that signature without disturbing a single strategy module, and the metrics log the engine already writes is precisely the training data such a model requires. The path from rule-based to learned selection is therefore an implementation of an existing interface, not a rewrite.

The second is the shift from per-page to per-component adaptation. Rendering strategies are currently applied to whole pages because that is the unit frameworks expose, but nothing in the engine requires it. A page is a component tree, and the strategy interface renders a component; a natural next step is a page whose navigation is statically generated, whose main content is streamed and whose interactive island is client-rendered, all decided per request. That direction aligns with where the wider ecosystem is already heading — islands architectures, partial hydration and server components — and an engine that already treats the strategy as a runtime variable is unusually well placed to adopt it.

Taken together, these directions describe a system that starts as a rule-based selector and grows into a self-tuning rendering controller: one that observes its own outcomes, learns which strategy actually served each class of visitor best, operates at component granularity across a distributed edge, and optimises not only for speed but for the cost and energy of producing a page. Every one of those steps is an extension of the seams built in this project rather than a departure from them, which is the strongest practical evidence that the architecture was the right one.

8. References

[1] Q. Wang, H. Shen, Y. Li, and Z. Zhang, “Anatomizing Deep Learning Inference in Web Browsers,” in Proc. Web Conf. 2021 (WWW ’21), Ljubljana, Slovenia, 2021, pp. 2819–2830, doi: 10.1145/3442381.3449890.

[2] M. Alshammari and R. Alshammari, “Comparison of Web Page Rendering Methods,” Int. J. Adv. Comput. Sci. Appl., vol. 13, no. 4, pp. 95–103, 2022.

[3] A. Kumar and S. Patel, “Server-Side vs Client-Side Rendering: Performance Evaluation Study,” Int. J. Comput. Appl., vol. 183, no. 12, pp. 1–8, 2021.

[4] J. Lee and K. Park, “The Impact of Server-Side Rendering on UI Performance and SEO,” J. Web Eng., vol. 21, no. 6, pp. 1503–1522, 2022.

[5] S. Ibrahim, T. Hassan, and M. Noor, “Web Rendering Strategies: A Comparative Analysis,” IEEE Access, vol. 11, pp. 45678–45692, 2023.

[6] R. Singh and P. Verma, “Experimental Analysis of Server-Side Caching for Web Performance,” ACM Trans. Web, vol. 14, no. 3, pp. 1–25, 2020.

[7] L. Chen, Y. Zhao, and M. Li, “Overview of Web Application Performance Optimization Techniques,” J. Syst. Softw., vol. 180, 111002, 2021.

[8] H. Rahman and D. Kim, “Frontend Performance Optimization in Modern Web Applications,” in Proc. Int. Conf. Software Engineering Advances, 2020, pp. 87–94.

[9] M. A. Hassan, S. Islam, and R. Rahman, “Improving Perceived Performance of Mobile Web Applications,” Mobile Inf. Syst., vol. 2019, Art. ID 7483956, 2019.

[10] N. Alotaibi and A. Alharbi, “Optimizing Web Interface Rendering for Mobile Applications with High User Traffic,” Int. J. Comput. Appl., vol. 185, no. 7, pp. 95–103, 2023.

[11] Y. Zhang, X. Wang, and J. Liu, “Mobility-Aware Edge Caching for Minimizing Latency in Vehicular Networks,” IEEE Trans. Veh. Technol., vol. 70, no. 8, pp. 8321–8334, 2021.

[12] A. R. Mahmoud, “Rendering Strategies and User Experience Analysis,” M.S. thesis, Dept. Comput. Sci., Univ. of Technology, 2022.

[13] S. Khalid and M. Usman, “Technologies for Modern Web Application Architecture,” J. Distrib. Syst. Web Technol., vol. 5, no. 2, pp. 44–58, 2023.

[14] React. Server-Side APIs: renderToString and renderToPipeableStream. Retrieved on 9 August 2026 from https://react.dev/reference/react-dom/server

[15] Node.js. HTTP and Stream API documentation, Node.js v20 LTS. Retrieved on 9 August 2026 from https://nodejs.org/docs/latest-v20.x/api/http.html

[16] Docker Inc. Docker Compose specification. Retrieved on 9 August 2026 from https://docs.docker.com/compose/compose-file/

[17] F5 NGINX. NGINX reverse proxy documentation. Retrieved on 9 August 2026 from https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

[18] Apache Software Foundation. ab — Apache HTTP server benchmarking tool. Retrieved on 9 August 2026 from https://httpd.apache.org/docs/2.4/programs/ab.html

Appendices

Appendix A: Sample Decision Log

The engine emits three lines per request. This is the primary evidence that the decision reported in the response header is the decision the engine actually took.

```
[ARE] Request: /dynamic
[ARE] Context: net=fast device=desktop cache=fresh load=low volatility=realtime heavy=false edge=false
[ARE] Strategy selected: CSR - Realtime data on a capable client -> fully interactive (CSR)
```

Appendix B: Repository Structure

```
src/
  core/         types.ts, context-analyzer.ts, decision-engine.ts,
                strategy-registry.ts, engine.ts
  config/       engine.config.ts, strategy-rules.ts, thresholds.ts
  strategies/   ssg/  ssr/  streaming-ssr/  isr/  csr/  edge-isr/
  cache/        cache-manager.ts, memory-cache.ts, file-cache.ts,
                redis-cache.ts, invalidation.ts
  metrics/      metrics-collector.ts, report-generator.ts, timing.ts,
                resource-usage.ts
  server/       server.ts, router.ts, middleware.ts
  simulation/   network-throttler.ts, device-profiler.ts, traffic-simulator.ts
  frontend/     pages/ (static, dynamic, heavy), components/, client/
docker/         Dockerfile, nginx/proxy.conf, nginx/Dockerfile
scripts/        build-client.ts, build-ssg.ts, generate-report.ts,
                switch-test.sh, verify-headers.sh, load-test.sh
tests/          decision-engine, context-analyzer, rendering, cache
experiments/    results/ (raw NDJSON metrics, aggregated JSON and CSV)
diagrams/       system architecture, decision flow, rendering pipeline, data flow
docs/           project context and the numbered design documents
docker-compose.yml
```

Appendix C: Reproducing the Experiments

Every result in Chapter 5 can be regenerated with the following commands from a clean checkout. The stack must be rebuilt rather than restarted after a code change, because the deployment mounts no volumes.

```
# 1. bring up the five-container private server from a clean state
docker compose down && docker compose up -d --build

# 2. correctness: unit tests and the strategy-switch matrix
npm test                                  # 28 tests, 4 files
bash scripts/switch-test.sh               # same URL, varied context

# 3. read the decision for any context (header form and query form)
curl -sI "localhost:8080/dynamic?net=fast&device=desktop"   | grep -i x-rendering-strategy
curl -sI -H 'X-Device-Type: mobile' localhost:8080/dynamic  | grep -i x-rendering-strategy

# 4. prove that streaming is not buffered (GET, not HEAD)
curl -s -D- -o /dev/null localhost:8080/heavy | grep -iE 'transfer-encoding|x-streaming'

# 5. edge behaviour (the default route is stamped as origin by the proxy)
```

```
curl -sI "localhost:8080/edge1/static?cache=cold"  | grep -i x-rendering-strategy
curl -sI "localhost:8081/static?cache=cold"        | grep -i x-rendering-strategy

# 6. load comparison: fixed SSR policy versus adaptive selection
ab -n 800 -c 50 "localhost:8080/dynamic?net=fast&volatility=realtime&device=mobile&load=low"
ab -n 800 -c 50 "localhost:8080/dynamic?net=fast&device=desktop"

# 7. aggregate the per-request metrics into per-strategy summaries
npm run report                             # experiments/results/report.{json,csv}
```

Appendix D: The Decision Policy as Implemented

The complete policy, reproduced from src/config/strategy-rules.ts. The table is data; the evaluator that consumes it is the nine-line function shown in Section 4.3.3.

```
export const STRATEGY_RULES: StrategyRule[] = [
  { test: (c) => c.volatility === 'static' && c.cacheState !== 'cold',
    strategy: 'SSG',            reason: 'Static content with usable cache' },
  { test: (c) => c.volatility === 'static' && c.isEdge,
    strategy: 'EDGE_ISR',       reason: 'Static content at the edge' },
  { test: (c) => c.load === 'high',
    strategy: 'ISR',            reason: 'High load - shed origin work' },
  { test: (c) => c.volatility === 'realtime' && c.networkSpeed === 'fast'
                                          && c.device === 'desktop',
    strategy: 'CSR',            reason: 'Realtime data on a capable client' },
  { test: (c) => c.volatility === 'realtime' && c.device === 'mobile',
    strategy: 'SSR',            reason: 'Realtime data on a weak device' },
  { test: (c) => c.volatility === 'periodic',
    strategy: 'ISR',            reason: 'Periodic data - cache and revalidate' },
  { test: (c) => c.heavyPayload && c.networkSpeed !== 'slow',
    strategy: 'STREAMING_SSR',  reason: 'Large payload on a decent link' },
  { test: (c) => c.networkSpeed === 'slow',
    strategy: 'SSR',            reason: 'Slow network - avoid heavy hydration' },
  { test: () => true,
    strategy: 'SSR',            reason: 'Fallback - safe, correct default' },
];
```

Appendix E: Per-Request Metric Record

One newline-delimited JSON record is appended per request. All quantitative results in Chapter 5 derive from this schema.

```
{
  "ts": "2026-08-09T15:12:44.310Z",
  "url": "/dynamic?net=fast&device=desktop",
  "strategy": "CSR",
  "reason": "Realtime data on a capable client -> fully interactive (CSR)",
  "ttfbMs": 0.04, "renderMs": 0.03, "totalMs": 0.07,
  "fromCache": false, "bytes": 853,
  "network": "fast", "device": "desktop", "load": "low", "isEdge": false,
  "resources": { "rssMb": 88.9, "heapUsedMb": 11.2, "cpuMs": 1462, "loadAvg1m": 2.31 }
}
```